# Steering Copilot

Design and Verify a Synchronous FIFO

**Tools:** VS Code + GitHub Copilot (inline autocomplete only)

## What Is a FIFO?

A FIFO (first in, first out) is a buffer that stores data and returns it in the same order it arrived. It sits between a *producer* that writes data and a *consumer* that reads it, letting the two run at different rates for short periods without losing data. FIFOs appear throughout digital systems: network interfaces, video pipelines, and anywhere one block produces data in bursts that another block consumes steadily.

A *synchronous* FIFO uses a single clock for both reading and writing. It is usually built from a small memory array used as a circular buffer, with a write pointer marking where the next entry goes and a read pointer marking the oldest entry. Status flags tell the producer when the FIFO is *full* (stop writing) and the consumer when it is *empty* (nothing to read). The interesting design work is in the boundary cases: what should happen when someone writes to a full FIFO, reads from an empty one, or does both at the same time?

**Today's goal** is not just a working FIFO. It is to find out how *the way you interact with Copilot* changes what it gives you. You will try to use just the inline capability (no Chat panel, no inline chat `Ctrl+I`, no agent mode, no other AI tools) to see what you can get. This style does not consume many usage tokens, so it is essentially free for students. Agentic modes will consume more tokens, and you may run into usage limits, so you can use it at your own discretion. Remember, you own every line you accept. If a suggestion disagrees with the spec, the suggestion is wrong.

**Before class:** Copilot installed and signed in. Next Edit Suggestions disabled in `settings.json`:
`"github.copilot.nextEditSuggestions.enabled": false`
Starter file `tb_skeleton.sv` downloaded.

## Copilot Steering Techniques

| | Technique | How |
|---|---|---|
| T1 | Zero context | Empty file, minimal typing, accept whatever comes. |
| T2 | Spec as header comment | Paste the spec as a comment at the top of the file. |
| T3 | Comment first | Write a comment describing the next block, then start a new line. |
| T4 | First line first | Type the opening line of a block yourself (no comment) and let it continue. |
| T5 | Compare alternatives | `Alt+]` / `Alt+[` to cycle, or `Ctrl+Enter` to see several at once. |
| T6 | Partial accept | `Ctrl+Right Arrow` accepts one word at a time. |
| T7 | Open-tab context | Copilot reads your open tabs. Compare suggestions with the DUT tab open vs. closed. |
| T8 | Naming as a prompt | Type a descriptive name, e.g. a task called `test_rw_when_full`, and let it fill the body. |
| T9 | Example, then pattern | Write one instance by hand, then let Copilot produce the next ones. |

## The Specification

```
module sync_fifo #(
  parameter int WIDTH = 8,
  parameter int DEPTH = 16   // always a power of 2, >= 2
)(
  input  logic             clk,
  input  logic             rst_n,     // asynchronous, active-low
  input  logic             wr_en,
  input  logic [WIDTH-1:0] wr_data,
  input  logic             rd_en,
  output logic [WIDTH-1:0] rd_data,
  output logic             full,
  output logic             empty,
  output logic [?:0]       count,     // entries stored; you choose the width
  output logic             overflow,  // 1-cycle pulse
  output logic             underflow  // 1-cycle pulse
);
```

1. **Reset:** empties the FIFO. It will also set `count = 0`, `rd_data = 0`, `overflow = underflow = 0`.
2. **Write:** on a rising edge, if `wr_en` and not full, store `wr_data`.
3. **Read:** on a rising edge, if `rd_en` and not empty, the oldest entry appears on `rd_data` (registered, so one cycle after `rd_en`). `rd_data` holds its value when no read occurs.
4. **Write while full** (no read): ignored; `overflow` pulses for one cycle.
5. **Read while empty:** ignored; `underflow` pulses for one cycle.
6. **Read + write while full:** both succeed; `count` stays `DEPTH`; no overflow.
7. **Read + write while empty:** only the write succeeds; `underflow` pulses; `count` becomes 1.
8. `count` represents the number of valid entries stored and must be wide enough to represent 0 through `DEPTH` inclusive.
9. `full` and `empty` are combinational signals from current state.

## Round 0: Predict

Read the spec. Make a prediction on which spec rule you think Copilot is most likely to get wrong.

## Round 1: Zero Context [T1]

Create `round1.sv`. Type only `module sync_fifo` and keep pressing `Tab`, accepting everything. Then spend two minutes checking the result against spec rules 4–9. Did it get everything right? How does this code look to you?

## Round 2: Design [T2] [T3] [T4] [T5]

Create `sync_fifo.sv` and paste the spec as a header comment **[T2]**. Build the design block by block. Along the way, run these two experiments:

- **Experiment A: comment first vs. first line first.** For the logic that decides whether a write happens, first write only a comment describing rules 4 and 6 and see what Copilot suggests **[T3]**. Delete it. Then type just the first line of the logic with no comment and see what it suggests **[T4]**. Did using either of these methods match the spec?

- **Experiment B: alternatives.** At one point in the design, press `Ctrl+Enter` **[T5]** and look at at least three suggestions. Were all alternatives correct?

Finish the design. You've learned the coding style expectations in this class. Did the generated code follow it? Can you wrangle it to follow better style? *Steering tip for the coding style:* a comment that names a register, its reset value, and its update condition (for example, `// Write pointer: reset to 0, increments on a successful write`) is a strong **[T3]** prompt. Without it, Copilot tends to put every register in one large block. A few quick checks you should do on the design before we move on to the testbench: do we have one `always_ff` per register group (different reset or enable ⇒ different block), nonblocking assignments, async active-low reset, `count` wide enough for `DEPTH`, rules 6 and 7 correct, single-cycle pulses.

## Round 3: Testbench [T6] [T7] [T8] [T9]

Copy `tb_skeleton.sv` and rename it `tb_sync_fifo.sv`. The clock, DUT instance, checker, coverage, driver tasks, and summary are provided. You fill in the three sections marked `YOUR CODE`: the reference model, at least one more assertion, and the directed tests.

- **Experiment C: open-tab context [T7].** With `sync_fifo.sv` open in another tab, start typing the reference model and look at the suggestion (don't accept it yet). Close the DUT tab and try again. Which suggestion looks more like a copy of your design? A model that copies the design shares its bugs, so write the model *from the spec* using the queue.
- **Naming [T8]**: write at least one directed test as a task with a descriptive name and let Copilot fill in the body.
- **Example, then pattern [T9]**: write one directed test by hand, then let Copilot generate the next ones.
- **Partial accept [T6]**: use word-by-word accept at least once on a suggestion that was only partly right.

Run the testbench on your design. It must print `RESULT: PASS`. The summary fails if your directed tests never reach full, overflow, underflow, or either read + write case. If you have a lot of time remaining, try adding 1,000+ cycles of random traffic.

## Round 4: Bug Hunt

There are 5 FIFO variants posted, `sync_fifo_bug1.sv` through `sync_fifo_bug5.sv`. Run your *unchanged* testbench against each one and see which it caught. Then pick one bug that escaped (or the one you found hardest to explain) and use any technique to get Copilot to write a test that catches it.

## Final Thoughts & Takeaways:

This exercise introduced you to some ways Copilot can help you with coding. You should now have a better idea of what types of things it can help with and what it may not be so good at. You as the designer will still have to do most of the heavy lifting, but Copilot can help get you to complete code faster (and probably save you from silly typos). It is up to you whether you would like to use AI for this class. Remember that you are responsible for every line of code you turn in, and are required to disclose if you used AI to complete an assignment (notes in code comments are fine). Regardless of AI usage, you are still required to be able to write code with pen and paper.